Written Homework 2: Computer Organization

CS 255: Computer Architecture and Machine Level Programming

Name: Solution NetID: _____

- **Objective**: This assignment provides guided practice reasoning about C programming concepts.
- **Submission**: Upload a completed copy of this PDF to Gradescope.
- **Grading**: Each problem is 10 points and will be graded using a check system. Showing your work is strongly encouraged.
  - Check-plus (100%): Correct solution with clear and complete reasoning.
  - Check (90%): Mostly correct, with minor error(s) or omission(s).
  - Check-minus (70%): Partially correct, with demonstrated understanding but significant mistakes or incomplete work.
  - No credit (0%): No meaningful attempt or a major conceptual misunderstanding.

1. *[Computer Overview]* Consider the following C program:

```
int x;
int y;

scanf("%d", &x);
y = x + 10;
printf("%d\n", y);
```

For each event below, identify the main component(s) involved and describe what happens to the data.

Event	**Component(s)**	**What happens to the data?**
the user enters 25	input device	the keyboard converts user input into data and sends it to memory
25 becomes the value of x	memory, CPU	the value 25 is stored in memory at the location x. CPU controls
x is obtained for the computation	CPU, memory	the value of x is read from memory into the CPU
x + 10 is computed	CPU	the CPU performs the addition
the result becomes the value of y	CPU, memory	the CPU stores the result 35 in the memory location y
the value of y is displayed	output device	the value is sent from memory to the display

2. *[Binary Representation]* Consider the 8-bit binary string `10110110`.

(a) Give the decimal value of the string when interpreted as an unsigned integer.

128 + 0 + 32 + 16 + 0 + 4 + 2 + 0 = 182

(b) Give the decimal value of the string when interpreted as a two's-complement signed integer.

-128 + 0 + 32 + 16 + 0 + 4 + 2 + 0 = -74

(c) Write the string in hexadecimal.

1011 0110

B 6 0xB6

(d) Sign-extend the string to 16 bits.

1111111110110110

(e) What decimal value does your answer to part (d) represent as a 16-bit two's-complement integer? Why?

-74

The leftmost 1 has a large negative place value (-2^{15}), but the additional leading 1s have positive place values which cancel out most of that negative value. The new leading bits plus the original MSB have the same value that the original MSB had.

3. *[Binary Representation]* Perform each of the following operations in binary. Show your binary arithmetic, the resulting 8-bit binary value and its decimal interpretation, and state whether overflow occurs and why. Assume all values are 8-bit two's-complement signed integers.

(a) `00110110 + 00011101`

```
  1 1 1 1
  00110110    54
+ 00011101  + 29
----------  ----
  01010011    83
```

no overflow - 83 is within the representable range

(b) `01100101 + 00101100`

```
  1 1   1 1
  01100101    101
+ 00101100  + 44
----------  ----
  10010001    -111
```

overflow - 145 is outside the representable range

(c) `10110110 + 11100101`

```
  1 1     1
  10110110    -74
+ 11100101  + -27
----------  ----
  10011011    -101
```

no overflow - -101 is within the representable range

4. *[Real Numbers]* Consider a 8-bit floating point representation with:

   - 1 sign bit,
   - 4 exponent bits (using excess-7 encoding), and
   - 3 fraction bits.

   Assume that this data type uses the conventions presented in class, including representations of zero, NaN, and (+/-)infinity.

(a) Represent the decimal value 0.8125 in this representation.

$0.8125 \times 2 = 1.625$
$0.625 \times 2 = 1.25$
$0.25 \times 2 = 0.5$
$0.5 \times 2 = 1.0$

fixed point = 0.1101

sci. not. = $(+)\,1.\boxed{101} \times 2^{\boxed{-1}}$ (S: +, F: 101, E: -1)

encoding = value + bias
$= -1 + 7 = 6_{10}$
$= 0110_2$

$\underset{S}{0}\ \underset{E}{0110}\ \underset{F}{101}$

(b) What decimal value does the bit string 11010011 represent?

$\underset{S}{1}\ \underset{E}{1010}\ \underset{F}{011}$

value = encoding − bias
$= 1010_2 - 7_{10}$
$= 10_{10} - 7_{10} = 3$

sci. not. = $(-)\,1.\boxed{011} \times 2^{\boxed{3}}$ (S: −, F: 011, E: 3)

fixed point = $-1011_2 = -(8+2+1) = -11_{10}$

5. *[Real Numbers]* Answer the following questions using the same floating point representation as the previous question.

(a) What is the largest positive representable value, other than infinity, of this representation?

max exponent : 1110 value = encoding - bias = 14 - 7 = 7

max mantissa : 1.111

$1.111 \times 2^7 = 11110000_2 = 240_{10}$

(b) What is the smallest positive representable value, other than zero, of this representation?

min exponent: 0001 value = encoding - bias = 1 - 7 = -6

min mantissa: 1.000

$1.000 \times 2^{-6} = 0.000001_2 = \frac{1}{64_{10}}$

(c) What is the step size (difference between adjacent representable values) between 1.0 and 2.0?

Between 1.0 and 2.0, the exponent is 0. With the exponent fixed, only the fraction bits distinguish values so there are 8 distinct values between 1.0 and 2.0. The values are evenly spaced, so the step size is $\frac{1}{8}$.

6. *[Memory]* Consider the following declarations:

```
struct Record {
    short values[3];
    int id;
    char flag;
};
struct Record records[4];
```

(a) Draw the memory layout of `records[0]`, clearly indicating padding and the end of the struct. Assume that `&records = 0x1000`.

	0x0	0x1	0x2	0x3
0x1000	values[0]	values[0]	values[1]	values[1]
0x1004	values[2]	values[2]	(pad)	(pad)
0x1008	id	id	id	id
0x100c	flag	(pad)	(pad)	(pad)

sizeof(struct Record) = 16 bytes

(b) What is the starting address of `records[2]`?

&records[0] = 0x1000

&records[1] = 0x1010

&records[2] = 0x1020

(c) What is the address of `records[2].values[1]`?

0x1020 + 0x2 = 0x1022

7. *[Memory]* Consider a computer system with a 16-bit address bus and a 32-bit data bus.

(a) What is the maximum byte-addressable memory capacity?

16 bit address bus can uniquely identify

2^{16} addresses, so 2^{16} bytes

(b) Suppose the CPU reads a 4-byte int beginning at address 0x1008. How many memory accesses are required? What about a 4-byte int beginning at address 0x100A? Explain.

4 bytes = 32 bits

0x1008 is aligned : 1 access required

0x100A is not aligned : 2 accesses required

① read 0x1008 - 0x100B

② read 0x100C - 0x100F

(c) Suppose instead that the computer has a 16-bit data bus. How many memory accesses would be required to read an aligned 4-byte int? How does this compare with the original system?

16 bit bus = 2 bytes / transfer

$$\frac{4 \text{ bytes}}{2 \frac{\text{bytes}}{\text{access}}} = 2 \text{ accesses required}$$

2x as many memory accesses

8. *[CPU Structure]* An 8-bit register `status` stores several values using the following format:

mode	A	B	C
Bits: 7--3	2	1	0

The five-bit field `mode` represents an unsigned integer from 0 to 31.

Write a C expression for each of the following.

(a) Determine whether flag `B` is set.

flag B = bit 1

(status >> 1) & 1

(b) Set flag `A`.

flag A = bit 2

status | 00000100

(c) Extract the value of `mode` as an integer from 0 to 31.

status >> 3

9. *[Instruction Execution Cycle]* Consider the following instruction which shifts the value in R2 left by 2 bits and stores the result in R1.

```
shift R1, R2, 2
```

Describe what happens during each stage of the instruction execution cycle:

(a) Fetch

the instruction is read from memory at the address in the PC

(b) Decode

① control unit decodes instruction
- read reg R2
- write reg R1
- PC sequential update
- no memory access
- ALU left shift

② read R2 from register file

(c) Execute

ALU shifts the value from R2 left by 2 bits

(d) Memory

no memory access

(e) Writeback

ALU result written to R1

PC is updated to PC+4

10. *[Instruction Execution Cycle]* A program executes 600 million instructions on a 3 GHz processor with an average CPI of 2.0.

(a) How long does the program take to execute?

$$\frac{\text{exec. time}}{\text{prog.}} = \frac{\text{inst.}}{\text{prog.}} \cdot \frac{\text{cycles}}{\text{inst.}} \cdot \frac{\text{sec.}}{\text{cycle}}$$

$$= \frac{600 \times 10^6 \text{ inst}}{\text{prog}} \cdot \frac{2.0 \text{ cycles}}{\text{inst}} \cdot \frac{1 \text{ sec}}{3 \times 10^9 \text{ cycle}} = \frac{1200 \text{ sec}}{3000 \text{ prog}} = \frac{4}{10} \frac{\text{sec}}{\text{prog}}$$

(b) A processor improvement reduces the average CPI to 1.0, but requires the clock rate to decrease to 2 GHz. How long does the program take to execute on the improved processor?

$$\frac{600 \times 10^6 \text{ inst}}{\text{prog}} \cdot \frac{1 \text{ cycle}}{\text{inst}} \cdot \frac{1 \text{ sec}}{2 \times 10^9 \text{ cycle}} = \frac{600 \text{ sec}}{2000 \text{ prog}} = \frac{3}{10} \frac{\text{sec}}{\text{prog}}$$

(c) Calculate the speedup from the improvement.

$$\text{speedup} = \frac{\text{baseline}}{\text{optimized}} = \frac{\frac{4}{10}}{\frac{3}{10}} = \left(\frac{4}{10}\right)\left(\frac{10}{3}\right) = \frac{4}{3}$$

(d) Explain why reducing CPI does not necessarily reduce execution time.

Execution time depends on CPI, instruction count, and clock rate. When CPI decreases the total number of cycles decreases, but if each cycle takes longer the total execution time can still increase.

(In this case the lower clock rate isn't enough to counteract the benefit of a lower CPI).

11. *[Caches]* A designer is considering a cache hierarchy with an L1 cache with a 25% miss rate and 1 cycle access time backed by a larger L2 cache with a 7% miss rate and 5 cycle access time. The L2 cache is backed by memory with a 100 cycle access time.

(a) What is the average access time, in cycles, of the L2 cache?

L2 AMAT = L2 access time + (L2 miss rate)(mem. access time)

= 5 cycles + (0.07)(100 cycles)

= 5 + 7 = 12 cycles

(b) What is the average access time, in cycles, of the L1 cache?

L1 AMAT = L1 access time + (L1 miss rate)(L2 AMAT)

= 1 cycle + (0.25)(12 cycles)

= 1 + 3 = 4 cycles

(c) Suppose the L2 cache is removed, so the L1 cache is backed directly by main memory. How much slower would the average memory access time be?

L1' AMAT = L1 access time + (L1 miss rate)(mem. access time)

= 1 cycle + (0.25)(100 cycles)

= 1 + 25 = 26 cycles

$$\frac{\text{slower}}{\text{faster}} = \frac{26 \text{ cycles}}{4 \text{ cycles}} = 6.5 \times \text{slower}$$

12. *[Caches]* Consider a byte-addressable system with 12-bit addresses and a 64-byte direct-mapped cache. Each cache block holds 8 bytes.

(a) How many blocks can the cache hold?

$$\frac{64 \text{ byte cache}}{8 \text{ bytes/block}} = 8 \text{ blocks}$$

direct mapped, so 8 sets

(b) How many bits of a memory address are used for the byte offset, set index, and tag?

byte offset : $\log_2 8 = 3$ bits

set index : $\log_2 8 = 3$ bits

tag : 12 bits − 3 bits − 3 bits = 6 bits

(c) Determine the tag, set index, and outcome for each memory access. Assume the cache is initially empty.

Address	Tag	Set Index	Hit/Miss
0 0 0 0 0 0 | 0 0 0 | 0 0 0	0	0	miss (empty)
0 0 0 0 0 0 | 0 1 1 | 0 0 0	0	3	miss (empty)
0 0 0 0 0 0 | 0 0 0 | 0 1 1	0	0	hit (same tag + set)
0 0 0 0 0 1 | 0 0 0 | 0 0 0	1	0	miss (different tag)
0 0 0 0 0 0 | 0 0 0 | 0 0 0	0	0	miss (different tag)
0 0 0 0 0 0 | 0 1 1 | 0 0 0	0	3	hit (same tag + set)

tag set byte